

+

Informatik 2 – Objektorientiertes Programmieren (OOP)

K

A



©image: various

Heute

- Was haben wir bereits?
- Wozu OOP?
- Grundlagen OOP
- Prozedural vs. OOP

C++

- Klassendefinition
- Überladung von Funktionen
- Modulare Programmierung

Nächstes Mal

- OOP (Teil 2)

C++

- Kommunikation
- Vererbung
- Polymorphie

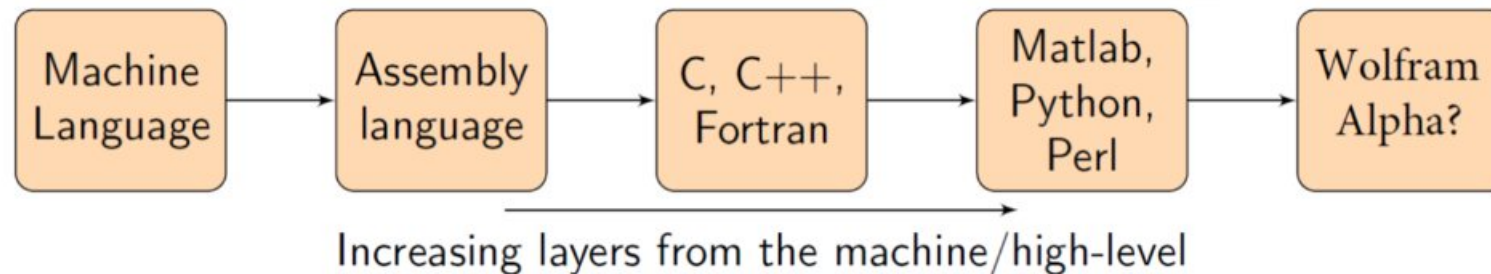
Wiederholung: Programmiersprachen



Machine language/code is the lowest-level programming paradigm, which directly represents the instructions (the contents of program memory) as a sequence of numbers.

Assembly language is a representation of machine code. In other words, each assembly language instruction translates to a machine code instruction. Though assembly language statements are readable, the statements are still low-level. A disadvantage of assembly language is that it is not portable, because each platform comes with a particular Assembly Language.

High-level languages are what most programmers use nowadays. Languages such as C, C++, Python and Java are all high-level languages. Advantages of high-level languages are that they are very readable and portable. Programs written in high-level languages are either compiled and/or interpreted into machine code so that computers can execute them.



Wiederholung: Programmiersprachen



In **Procedural Languages** a program is written in a sequences of steps that should be followed to produce a result. E.g. COBOL, FORTRAN and C

In **Object Oriented Programming (OOP) languages** a program is divided into object that contain data as well as methods that operate on the data. E.g. Python, Java, C# and C++.

In **Functional languages** a program is divided into function definitions. Functional languages are a kind of declarative language. E.g. Scala, F#, Clojure and Lisp.

Logic / Declarative Programming languages are used to create programs that enables the computer to reason logically. Specification of the task instead of the algorithmic model (“Was statt wie”).



Wiederholung: Programmiersprachen



Informatik 1

In **Procedural Languages** a program is written in a sequences of steps that should be followed to produce a result. E.g. COBOL, FORTRAN and C

In **Object Oriented Programming (OOP) languages** a program is divided into object that contain data as well as methods that operate on the data. E.g. Python, Java, C# and C++.

In **Functional languages** a program is divided into function definitions. Functional languages are a kind of declarative language. E.g. Scala, F#, Clojure and Lisp.

Logic / Declarative Programming languages are used to create programs that enables the computer to reason logically. Specification of the task instead of the algorithmic model (“Was statt wie”).



Wiederholung: Programmiersprachen



In **Procedural Languages** a program is written in a sequences of steps that should be followed to produce a result. E.g. COBOL, FORTRAN and C

In **Object Oriented Programming (OOP) languages** a program is divided into object that contain data as well as methods that operate on the data. E.g. Python, Java, C# and C++.

bereits benutzt?

In **Functional languages** a program is divided into function definitions. Functional languages are a kind of declarative language. E.g. Scala, F#, Clojure and Lisp.

Logic / Declarative Programming languages are used to create programs that enables the computer to reason logically. Specification of the task instead of the algorithmic model (“Was statt wie”).



Vergleich: Prozedurale Programmierung vs. OOP



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data = 0 ;
    Node* next = nullptr;
};
```

```
void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next; }}
}
```

```
void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next; }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;}
```

```
int main() {
    Node* Anker = nullptr;
    Node* temp = Anker;
    for (int jj = 0; jj < 20; jj++) {
        Node* nE = new Node;
        nE->data = jj;
        if (temp != nullptr) {
            temp->next = nE;
            temp = nE;}
        else {
            Anker = nE;
            temp = nE;}
    }
    AusgabeListe(Anker);
}
```

```
// Löschen des nten Elements
int n;
cout << "Welches Element soll
        geloescht werden?\n";
cin >> n;
```

```
LoescheElement(Anker,n)
```

```
AusgabeListe(Anker);
}
```

Procedural

OOP



Vergleich: Prozedurale Programmierung vs. OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0 ;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next; }

    void LoescheElement(Node* Start,
                        int welches) {
        Node* temp = Start;
        for (int jj = 0; jj < n - 1; jj++) {
            temp = temp->next; }
        Node* help = temp->next;
        temp->next = help->next;
        delete help;}
```

```
int main() {
    Node* Anker = nullptr;
    Node* temp = Anker;
    for (int jj = 0; jj < 20; jj++) {
        Node* nE = new Node;
        nE->data = jj;
        if (temp != nullptr) {
            temp->next = nE;
            temp = nE;}
        else {
            Anker = nE;
            temp = nE;}
    }
    AusgabeListe(Anker);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
            gelöscht werden?\n";
    cin >> n;

    LoescheElement(Anker,n)

    AusgabeListe(Anker);
}
```

Procedural

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
            gelöscht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP



Vergleich: Prozedurale Programmierung vs. OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next; }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next; }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;}
```

```
int main() {
    Node* Anker = nullptr;
    Node* temp = Anker;
    for (int jj = 0; jj < 20; jj++) {
        Node* nE = new Node;
        nE->data = jj;
        if (temp != nullptr) {
            temp->next = nE;
            temp = nE;
        }
        else {
            Anker = nE;
        }
    }
}
```

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    ...
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    ...
}

// n Elements

int n;
cout << "Welches Element soll
        geloescht werden?\n";
cin >> n;

Liste.erase(Liste.begin() + n);

AusgabeListe(&Liste);
}
```

Was sind die Vorteile von OOP?

```
int n;
cout << "Welches Element soll
        geloescht werden?\n";
cin >> n;

LoescheElement(Anker,n)

AusgabeListe(Anker);
}
```

Procedural

OOP



Vorteil: OOP?



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0 ;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next; }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next; }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;}

int main() {
    Node* Anker = nullptr;
    Node* temp = Anker;
    for (int jj = 0; jj < 20; jj++) {
        Node* nE = new Node;
        nE->data = jj;
        if (temp != nullptr) {
            temp->next = nE;
            temp = nE;}
        else {
            Anker = nE;
            temp = nE;}
    }
    AusgabeListe(Anker);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
            gelöscht werden?\n";
    cin >> n;

    LoescheElement(Anker,n)

    AusgabeListe(Anker);
}
```

Procedural

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
            gelöscht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP



Vorteil: OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next;
    }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next;
    }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;
}
```

Procedural

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
        geloescht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP

Programmieren mit „Baukasten“

- reduzierte Komplexität
- mehr Komfort
- höherer Produktivität



Vorteil: OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next;
    }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next;
    }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;
}

int main() {
    Node* Anker = nullptr;
    Node* temp = Anker;
    for (int jj = 0; jj < 20; jj++) {
        Node* nE = new Node;
        nE->data = jj;
        if (temp != nullptr) {
            temp->next = nE;
            temp = nE;
        }
        else {
            Anker = nE;
            temp = nE;
        }
        AusgabeListe(Anker);
    }

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
        gelöscht werden?\n";
    cin >> n;

    LoescheElement(Anker, n);

    AusgabeListe(Anker);
}
```

Procedural

Programmieren mit „Baukasten“

- reduzierte Komplexität
- mehr Komfort
- höherer Produktivität



Erlaubt besseren Code

- weniger Fehler
- sicherere Code

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
        gelöscht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP



Vorteil: OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next;
    }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next;
    }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;
}
```

Procedural

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
        geloescht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP

Programmieren mit „Baukasten“

- reduzierte Komplexität
- mehr Komfort
- höherer Produktivität



Erlaubt besseren Code

- weniger Fehler
- sicherere Code

Kapseln von Programmelementen

- erleichtert das Zusammenarbeiten vieler Softwareentwickler
- ermöglicht geregelten/abgesicherten Zugriff



Vorteil: OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next;
    }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next;
    }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;
}
```

Procedural

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
    gelöscht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP

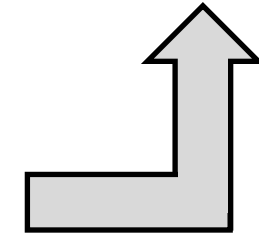
Programmieren mit „Baukasten“

- reduzierte Komplexität
- mehr Komfort
- höherer Produktivität



Erlaubt besseren Code

- weniger Fehler
- sicherere Code



Kapseln von Programmelementen

- erleichtert das Zusammenarbeiten vieler Softwareentwickler
- ermöglicht geregelten/abgesicherten Zugriff



Vorteil: OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next;
    }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next;
    }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;
}
```

Procedural

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
        geloescht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP

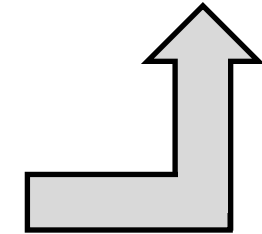
Programmieren mit „Baukasten“

- reduzierte Komplexität
- mehr Komfort
- höherer Produktivität



Erlaubt besseren Code

- weniger Fehler
- sicherere Code



Kapseln von Programmelementen

- erleichtert das Zusammenarbeiten vieler Softwareentwickler
- ermöglicht geregelten/abgesicherten Zugriff

Näher an der menschlichen Denkweise



Vorteil: OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next;
    }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next;
    }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;
}
```

Procedural

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
        geloescht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP

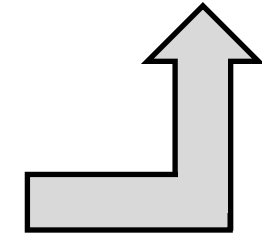
Programmieren mit „Baukasten“

- reduzierte Komplexität
- mehr Komfort
- höherer Produktivität



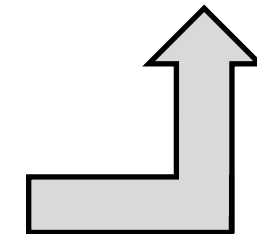
Erlaubt besseren Code

- weniger Fehler
- sicherere Code



Kapseln von Programmelementen

- erleichtert das Zusammenarbeiten vieler Softwareentwickler
- ermöglicht geregelten/abgesicherten Zugriff



Näher an der menschlichen Denkweise



Vorteil: OOP



```
#include <iostream>
using namespace std;

struct Node {
    int data = 0;
    Node* next = nullptr;
};

void AusgabeListe(Node* Start) {
    Node* temp = Start;
    while (temp != nullptr) {
        cout << temp->data << endl;
        temp = temp->next;
    }
}

void LoescheElement(Node* Start,
                    int welches) {
    Node* temp = Start;
    for (int jj = 0; jj < n - 1; jj++) {
        temp = temp->next;
    }
    Node* help = temp->next;
    temp->next = help->next;
    delete help;
}
```

Procedural

```
#include <iostream>
#include <vector>
using namespace std;

void AusgabeListe(vector<int>* welche) {
    for (int jj = 0; jj < welche->size(); jj++)
        cout << welche->at(jj) << endl;
}

int main() {
    vector<int> Liste;
    for (int jj = 0; jj < 20; jj++) Liste.push_back(jj);
    AusgabeListe(&Liste);

    // Löschen des nten Elements
    int n;
    cout << "Welches Element soll
        gelöscht werden?\n";
    cin >> n;

    Liste.erase(Liste.begin() + n);

    AusgabeListe(&Liste);
}
```

OOP

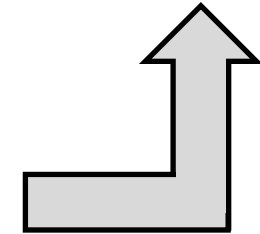
Programmieren mit „Baukasten“

- reduzierte Komplexität
- mehr Komfort
- höherer Produktivität



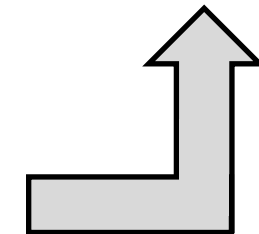
Erlaubt besseren Code

- weniger Fehler
- sicherere Code



Kapseln von Programmelementen

- erleichtert das Zusammenarbeiten vieler Softwareentwickler
- ermöglicht geregelten/abgesicherten Zugriff



Näher an der menschlichen Denkweise

Übergabe Programmablauf an Programmelemente

- ermöglicht Multitasking



Wie weit kommen wir mit Prozeduraler Programmierung?



Prozedurale Programmierung

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie `top()`, `pop()` und `push()` bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.

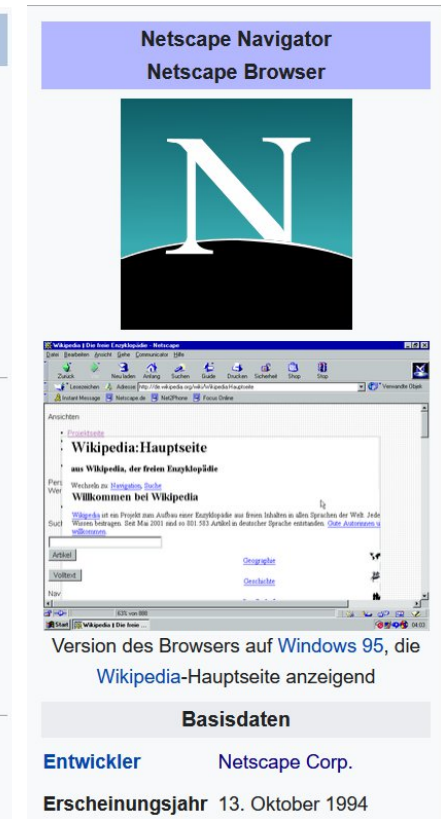
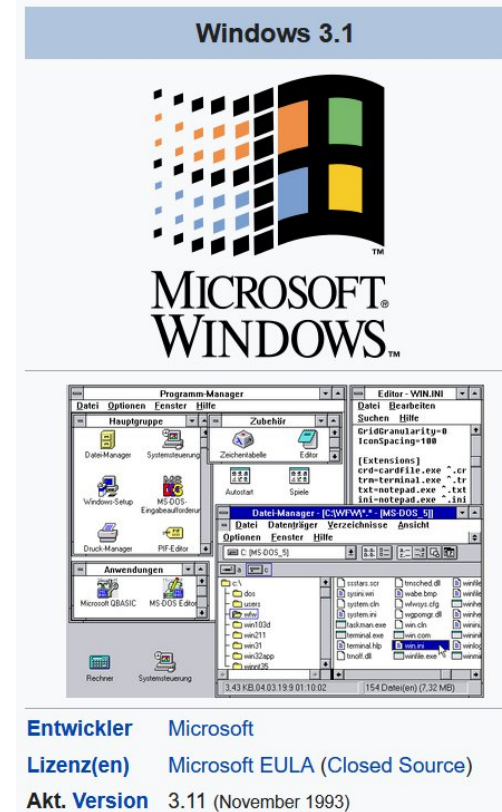
Wie weit kommen wir mit Prozeduraler Programmierung?



Prozedurale Programmierung

~1990er Jahre

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie `top()`, `pop()` und `push()` bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.



Wie weit kommen wir mit Prozeduraler Programmierung?

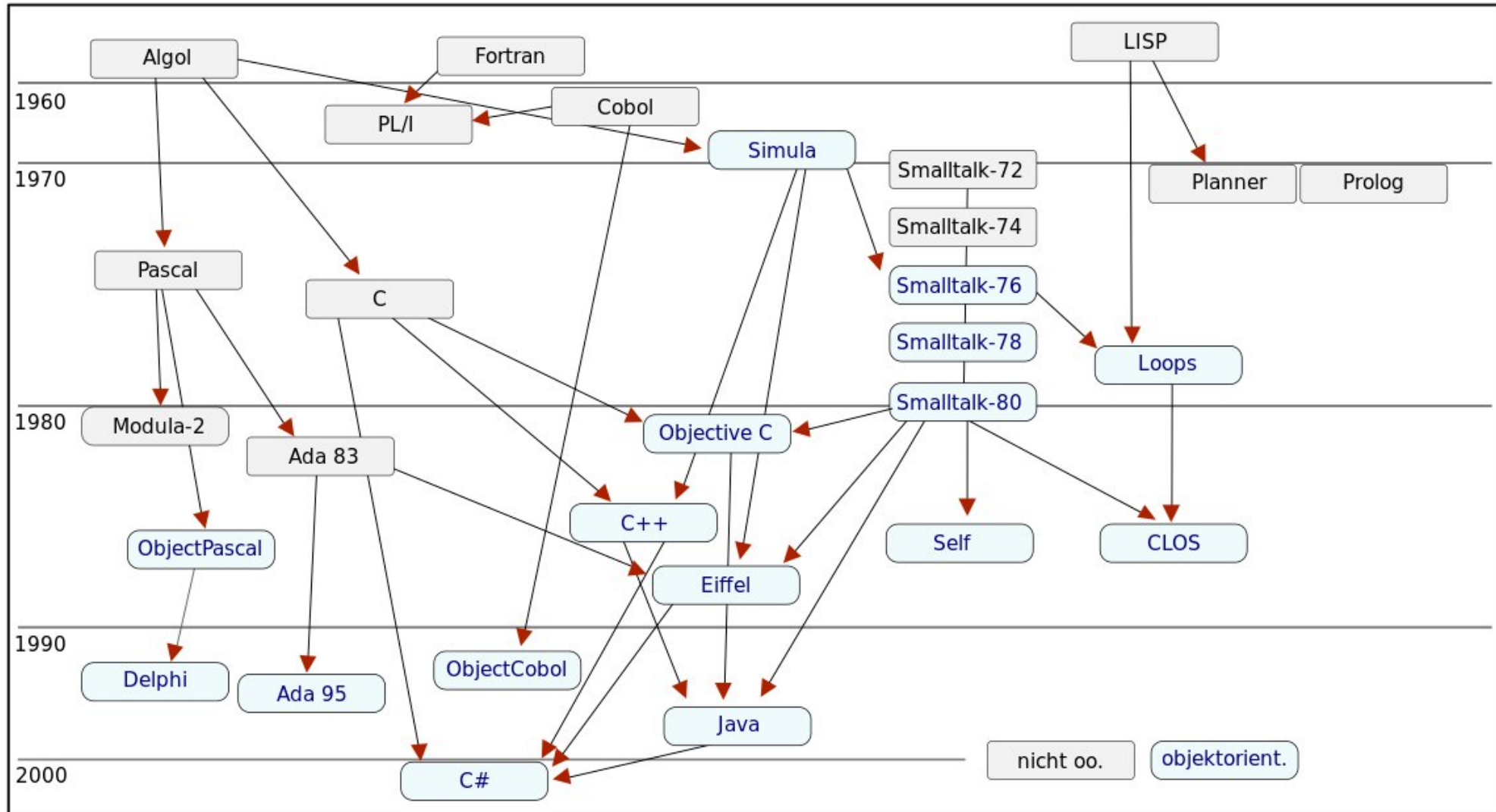


Prozedurale Programmierung

~1990er Jahre

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
 - Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
 - So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
 - Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie top(), pop() und push() bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.
- Tendenz zur Verwendung **vernetzter Geräte**: Verwendung von **grafischen Betriebssystemen** & Benutzeroberflächen, Netzwerk-Stacks (TCP/IP).
 - Notwendige Softwarefunktionalität ist bereits in Bibliotheken vorhanden und muss daher nicht mehr neu implementiert werden.
 - Verwendung dieser **Bibliotheken als „Baukastensystem“**: Bausteine sind Objekte, die auch dann verwendet werden können, wenn man deren Inhalt nicht kennt, sondern nur Merkmale und Funktionsweise

Geschichte OOP



Wiederholung: Programmiersprachen



In **Procedural Languages** a program is written in a sequences of steps that should be followed to produce a result. E.g. COBOL, FORTRAN and C

In **Object Oriented Programming (OOP) languages** a program is divided into object that contain data as well as methods that operate on the data. E.g. Python, Java, C# and C++.



In **Functional languages** a program is divided into function definitions. Functional languages are a kind of declarative language. E.g. Scala, F#, Clojure and Lisp.

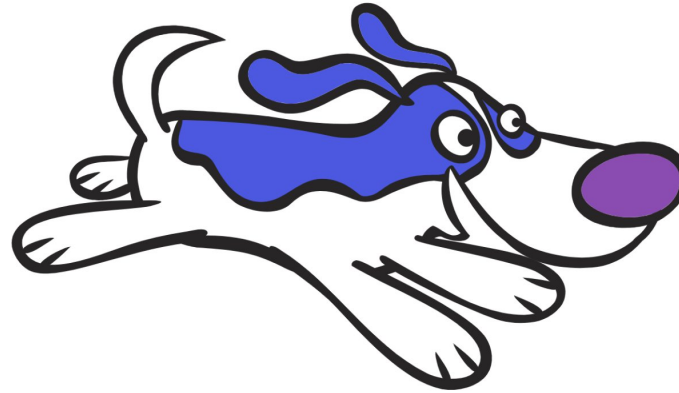
Logic / Declarative Programming languages are used to create programs that enables the computer to reason logically. Specification of the task instead of the algorithmic model (“Was statt wie”).



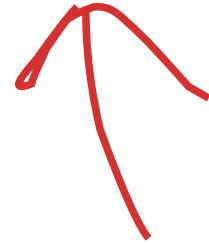
Haben wir schon einmal objektorientiert programmiert?



Haben wir schon einmal objektorientiert programmiert?



Haben wir schon einmal objektorientiert programmiert?



Objekt 1



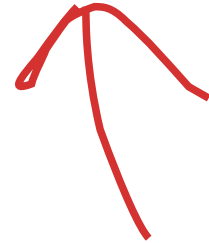
Objekt 2



In der Objektorientierten Programmierung:



1. Alles ist ein Objekt!



Objekt 1

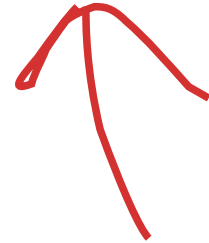


Objekt 2

In der Objektorientierten Programmierung:



1. Alles ist ein Objekt!
2. Objekte interagieren



Objekt 1

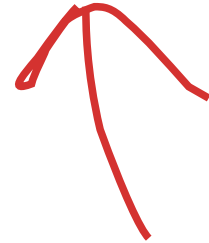


Objekt 2

In der Objektorientierten Programmierung:



1. Alles ist ein Objekt!
2. Objekte interagieren
3. Objekte haben eigenen
 - Speicher
 - Funktionen



Objekt 1

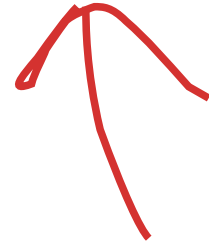


Objekt 2

In der Objektorientierten Programmierung:



1. Alles ist ein Objekt!
2. Objekte interagieren
3. Objekte haben eigenen
 - Speicher
 - Funktionen
4. Programmablauf wird an Objekte übergeben



Objekt 1



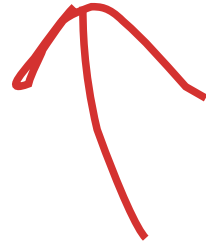
Objekt 2



In der Objektorientierten Programmierung:



1. Alles ist ein Objekt!
2. Objekte interagieren
3. Objekte haben eigenen
 - Speicher
 - Funktionen
4. Programmablauf wird an Objekte übergeben
5. Objekte sind Instanzen von Klassen



Objekt 1

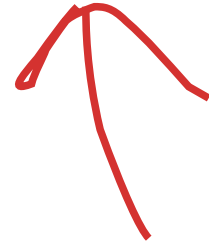


Objekt 2

In der Objektorientierten Programmierung:



1. Alles ist ein Objekt!
2. Objekte interagieren
3. Objekte haben eigenen
 - Speicher
 - Funktionen
4. Programmablauf wird an Objekte übergeben
5. Objekte sind Instanzen von Klassen



Objekt 1



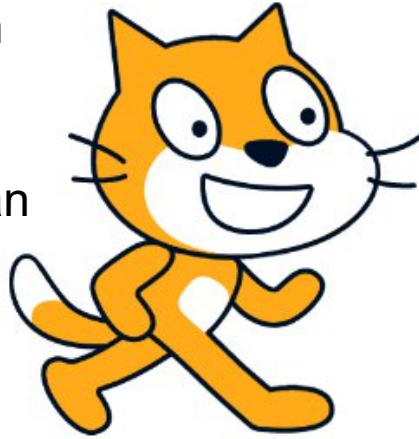
Objekt 2



In der Objektorientierten Programmierung:



1. Alles ist ein Objekt!
2. Objekte interagieren
3. Objekte haben eigenen
 - Speicher
 - Funktionen
4. Programmablauf wird an Objekte übergeben
5. Objekte sind Instanzen von Klassen



Objekt 1



Objekt 2



Objekt 3

1. Everything is an object
2. Objects communicate by sending and receiving messages (in terms of objects),
3. Objects have their own memory (in terms of objects),
4. Every object is an instance of a class (which must be an object),
5. The class holds the shared behavior for its instances (in the form of objects in a program list),
6. To eval a program list, control is passed to the first object and the remainder is treated as its message

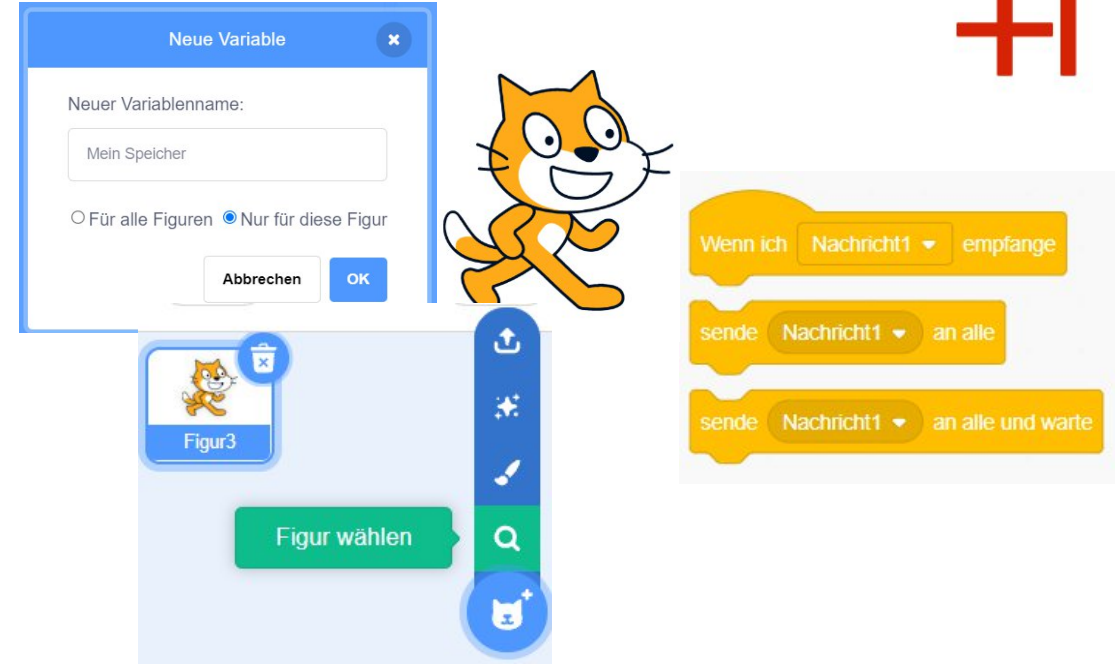
„1. Alles ist ein Objekt, 2. Objekte kommunizieren durch das Senden und Empfangen von Nachrichten (welche aus Objekten bestehen), 3. Objekte haben ihren eigenen Speicher (strukturiert als Objekte), 4. Jedes Objekt ist die Instanz einer Klasse (welche ein Objekt sein muss), 5. Die Klasse beinhaltet das Verhalten aller ihrer Instanzen (in der Form von Objekten in einer Programmliste), 6. Um eine Programmliste auszuführen, wird die Ausführungskontrolle dem ersten Objekt gegeben und das Verbleibende als dessen Nachricht behandelt“

– Alan Kay (Erfinder von Smalltalk & des Begriffs „object oriented“): The Early History of Smalltalk (1993)

https://de.wikipedia.org/wiki/Objektorientierte_Programmierung

OOP Definition

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class
5. The class holds the shared behavior for its instances
6. Program control is passed to the object

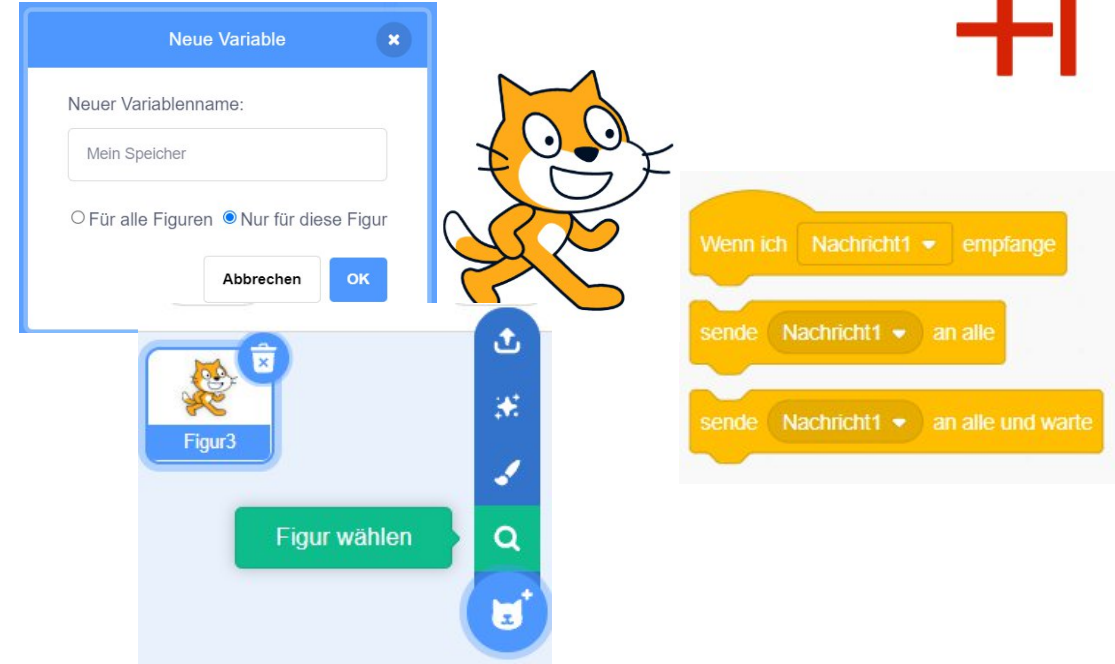


OOP Definition: Klassen

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class
5. The class holds the shared behavior for its instances
6. Program control is passed to the object

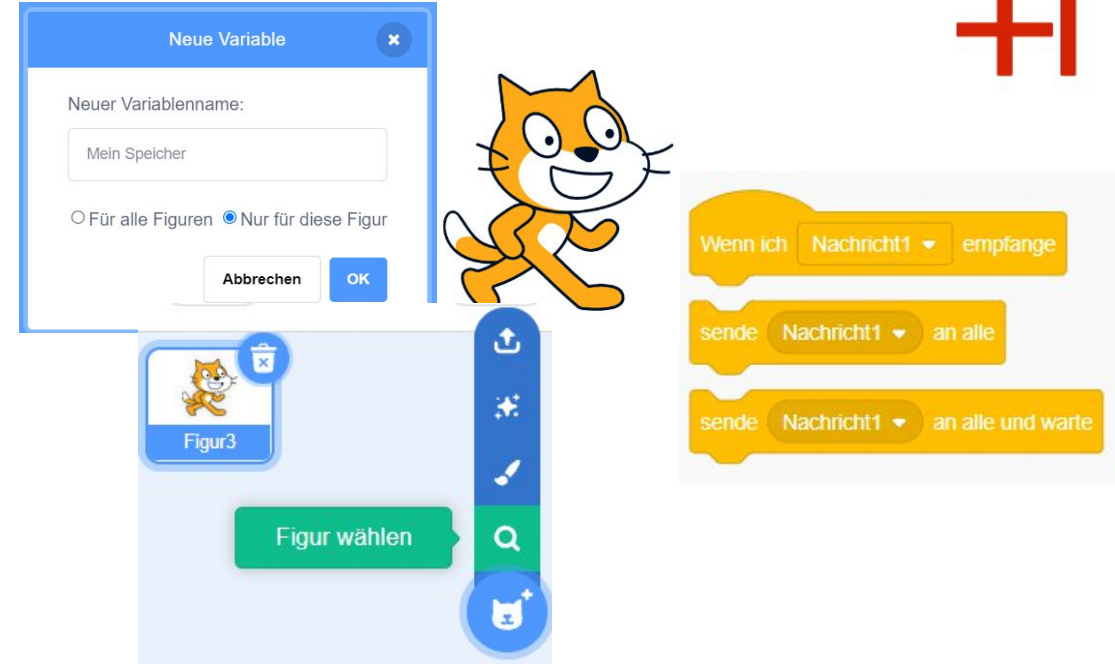
Allgemein: **Jedes Objekt im System** kann als ein **abstraktes Modell eines Akteurs** betrachtet werden,

- der Aufträge erledigen,
- seinen Zustand berichten und ändern
- und mit den anderen Objekten im System kommunizieren kann
- ohne offenlegen zu müssen, wie diese Fähigkeiten implementiert.



OOP Definition: Klassen

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class
5. The class holds the shared behavior for its instances
6. Program control is passed to the object



Allgemein: **Jedes Objekt im System** kann als ein **abstraktes Modell eines Akteurs** betrachtet werden,

- der Aufträge erledigen,
- seinen Zustand berichten und ändern
- und mit den anderen Objekten im System kommunizieren kann
- ohne offenlegen zu müssen, wie diese Fähigkeiten implementiert.

Solche Abstraktionen sind Klassen



Begriffsdefinition - Klasse



Definition (bitte merken!):

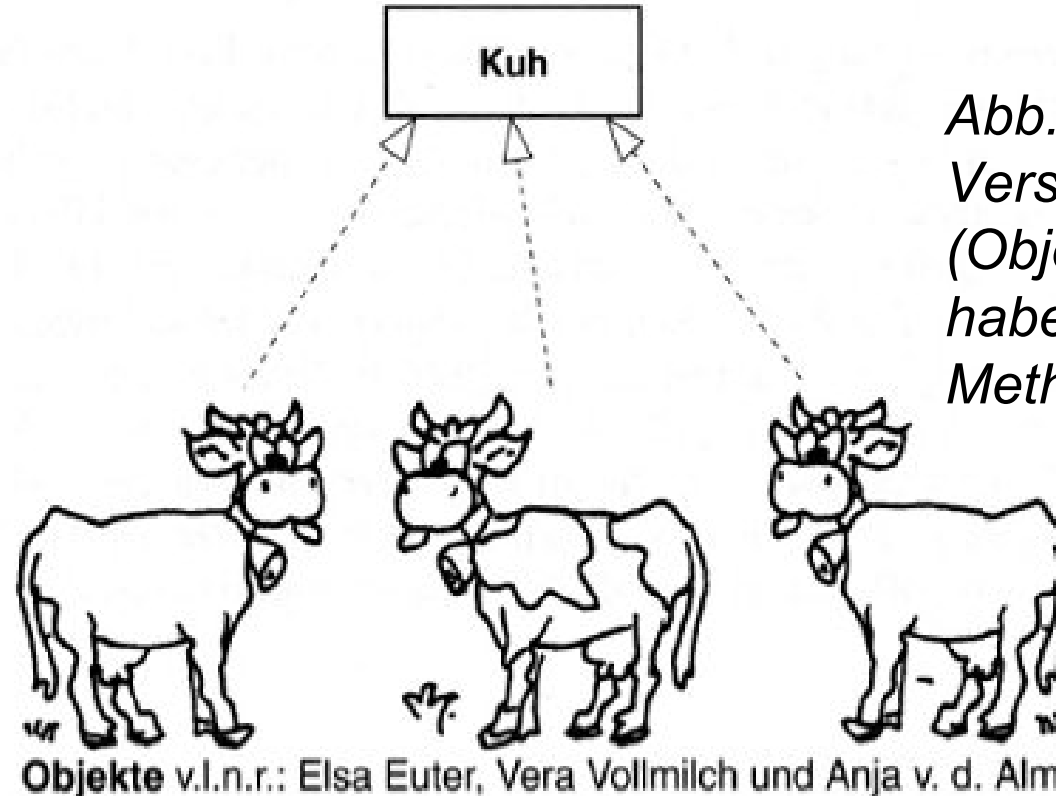
Sie ist ein **Bauplan** für Objekte! (im Speicher gibt es sie nicht).

Eine **Klasse** definiert die **Attribute und Methoden** einer Menge **gleichartiger Instanzen/Objekte**.

Attribute sind die **Daten** (Variablen) einer Klasse.

Methoden sind die **Funktionen** einer Klasse

Attribute und Methoden werden **Mitglied (engl. Member)** der Klasse genannt.



*Abb.:
Verschiedene Instanzen
(Objekte) der Klasse „Kuh“
haben die selben Attribute und
Methoden.*

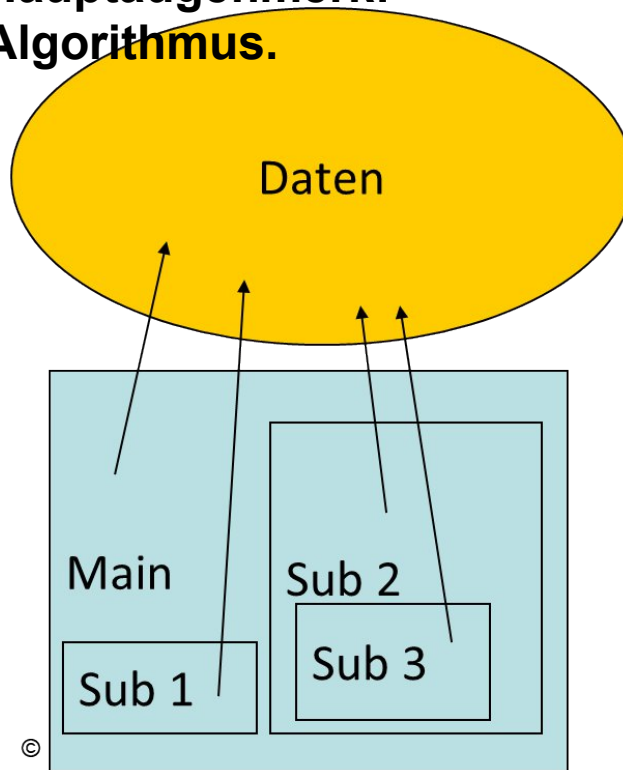
Quelle: Oesterreich (1998)

Vergleich prozedurale Programmierung – OOP



Daten und Funktionen
getrennt.

**Hauptaugenmerk:
Algorithmus.**



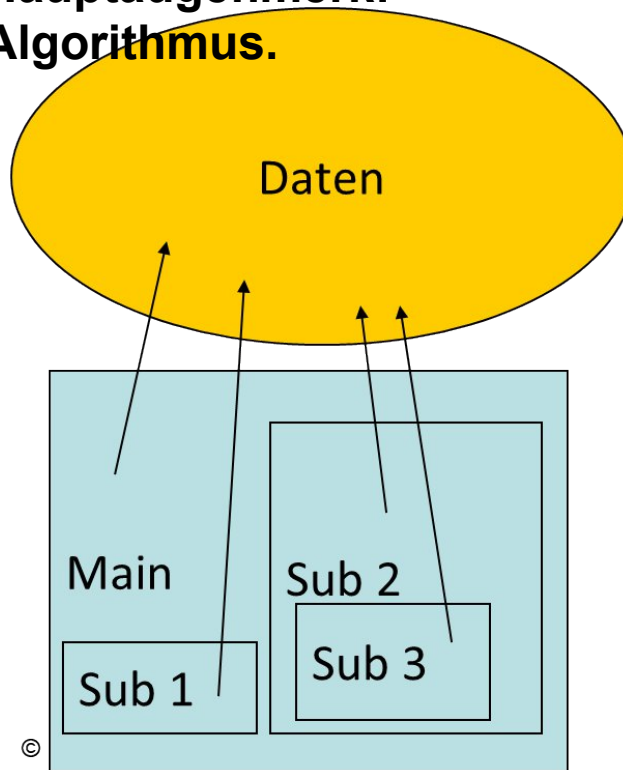
prozedurale Programmierung

Vergleich prozedurale Programmierung – OOP



Daten und Funktionen
getrennt.

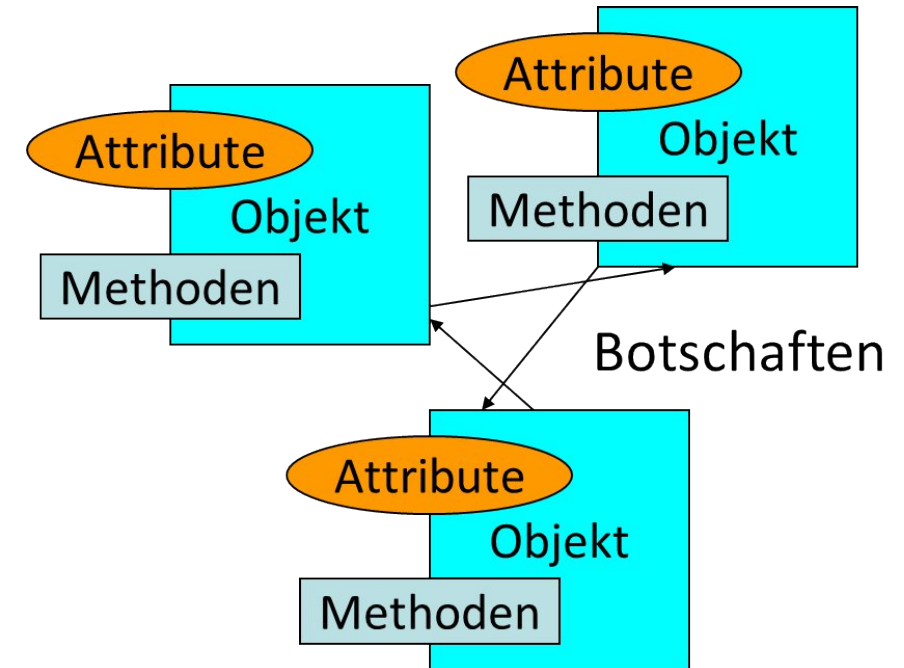
Hauptaugenmerk:
Algorithmus.



prozedurale Programmierung

Daten und Funktionen bilden Einheit
(Attribute und Methoden: gekapselt).

Hauptaugenmerk:
Kommunikation über Botschaften.



OOP



C++

Klassendefinition in C++



```
#include <iostream>
```

```
class Socke
```

```
{
```

```
public:
```

```
private:
```

```
};
```

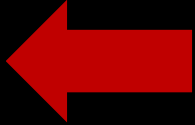


Klassendefinition in C++



```
#include <iostream>
```

```
class Socke  
{  
public:
```



Schlüsselwort class
Name: Socke

```
private:
```

```
};
```



Abschluss mit ;



Klassendefinition in C++

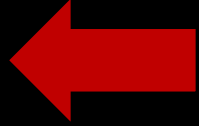


```
#include <iostream>
```

```
class Socke
```

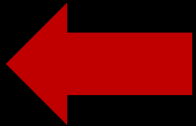
```
{
```

```
public:
```



Definition der
Zugriffsebene

```
private:
```



Definition der
Zugriffsebene

```
};
```

Zugriffstyp

Bedeutung

private

Klassenmember können nur von Memberfunktionen und Freunden (Klassen oder Funktionen) der Klasse verwendet werden.

protected

Klassenmember können von Memberfunktionen und Freunden (Klassen oder Funktionen) der Klasse verwendet werden. Darüber hinaus können sie von Klassen verwendet werden, die aus der Klasse abgeleitet sind.

public

Klassenmember können von beliebigen Funktion verwendet werden.

<https://learn.microsoft.com/de-de/cpp/cpp/member-access-control-cpp?view=msvc-170>



Objekt/Instanz wird instanziiert (erzeugt)



```
#include <iostream>
```

```
class Socke
```

```
{
```

```
public:
```

```
private:
```

```
};
```

```
int main(){  
    Socke MeineSocke;
```

```
    return 0;
```

```
}
```



Attribute



```
#include <iostream>

class Socke
{
public:

private:
    int age;           // sock age in years
    int size;          // size EU
    int value;         // in EUR
};
```

```
int main(){
    Socke MeineSocke;

    return 0;
}
```



Private Attribute der Klasse Socke



Konstruktor



```
#include <iostream>
```

```
class Socke
```

```
{
```

```
public:
```

```
    Socke()
```

```
{
```

```
    age = 0;
```

```
    size = 44;
```

```
    value = 10;
```

```
}
```

Konstruktor



```
private:
```

```
    int age;           // sock age in years
```

```
    int size;          // size EU
```

```
    int value;         // in EUR
```

```
};
```

```
int main(){
    Socke MeineSocke;

    return 0;
}
```

Der Konstruktor wird bei jeder Instanziierung aufgerufen.

Jede C++ Klasse muss zumindest einen Konstruktor und einen Destruktor besitzen, allerdings wird dieser vom Compiler ggf. automatisch erzeugt.



Konstruktor



```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

private:
    int age;           // sock age in years
    int size;          // size EU
    int value;         // in EUR
};
```

Konstruktor

Initialisierung der Attribute

```
int main(){
    Socke MeineSocke;

    return 0;
}
```

Der Konstruktor wird bei jeder Instanziierung aufgerufen.

Jede C++ Klasse muss zumindest einen Konstruktor und einen Destruktor besitzen, allerdings wird dieser vom Compiler ggf. automatisch erzeugt.



Destruktor

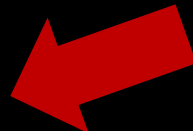


```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

    ~Socke()
    {
    }

private:
    int age;           // sock age in years
    int size;          // size EU
    int value;         // in EUR
};
```



Destruktor

```
int main(){
    Socke MeineSocke;

    return 0;
}
```

Der Destruktor wird bei Objektlöschung mit `delete` oder `delete [ ]` aufgerufen.

Jede C++ Klasse muss zumindest einen Konstruktor und einen Destruktor besitzen, allerdings wird dieser vom Compiler ggf. automatisch erzeugt.



Methods



```
#include <iostream>
```

```
class Socke
```

```
{
```

```
public:
```

```
    Socke()
```

```
{
```

```
    age = 0;
```

```
    size = 44;
```

```
    value = 10;
```

```
}
```

```
~Socke() {}
```

```
int get_age() {
```

```
    return age;
```

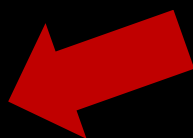
```
}
```

```
void set_age(int new_age) {
```

```
    age = new_age;
```

```
}
```

Public
Methods



```
private:
```

```
    int age;           // sock age in years
```

```
    int size;          // size EU
```

```
    int value;         // in EUR
```

```
};
```

```
int main(){
```

```
    Socke MeineSocke;
```

```
    return 0;
```

```
}
```



Methods



```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

    ~Socke() {}

    int get_age() {
        return age;
    }

    void set_age(int new_age) {
        age = new_age;
    }
}
```

```
private:
    int age;           // sock age in years
    int size;          // size EU
    int value;         // in EUR
};
```

```
int main(){
    Socke MeineSocke;
    cout << MeineSocke.get_age() << endl;
    MeineSocke.set_age(3);
    cout << MeineSocke.get_age() << endl;
```

```
    return 0;
```

```
}
```

public
Methods benutzen



Methods



```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

    ~Socke() {}

    int get_age() {
        return age;
    }

    void set_age(int new_age) {
        age = new_age;
    }
}
```

```
private:
    int age;        // sock age in years
    int size;       // size EU
    int value;      // in EUR
};
```

```
int main(){
    Socke MeineSocke;
    cout << MeineSocke.get_age() << endl;
    MeineSocke.set_age(3);
    cout << MeineSocke.get_age() << endl;
```

```
    return 0;
```

```
}
```

public
Methods benutzen

Haben wir bereits früher auf Objekt Instanzen
zugegriffen?



Zugriff auf private Attributes



```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

    ~Socke() {}

    int get_age() {
        return age;
    }

    void set_age(int new_age) {
        age = new_age;
    }
}
```

```
private:
    int age;           // sock age in years
    int size;          // size EU
    int value;         // in EUR
};

int main(){
    Socke MeineSocke;
    cout << MeineSocke.get_age() << endl;
    MeineSocke.set_age(3);
    cout << MeineSocke.get_age() << endl;

    MeineSocke.age = 1;

    return 0;
}
```

member "Socke::age" (declared at line 42) is inaccessible C/C++(265)

int Socke::age

sock age in years

[View Problem \(Alt+F8\)](#) [Quick Fix... \(Ctrl+.\)](#)

Zugriff auf private Attributes nur mit Getter/Setter Methods



```
#include <iostream>
```

```
class Socke
```

```
{
```

```
public:
```

```
    Socke()
```

```
    {
```

```
        age = 0;
```

```
        size = 44;
```

```
        value = 10;
```

```
    }
```

```
~Socke() {}
```

```
int get_age() {
```

```
    return age;
```

```
}
```

```
void set_age(int new_age) {
```

```
    age = new_age;
```

```
}
```

Getter



Setter



```
private:
```

```
    int age;           // sock age in years
```

```
    int size;          // size EU
```

```
    int value;          // in EUR
```

```
};
```

```
int main(){
```

```
    Socke MeineSocke;
```

```
    cout << MeineSocke.get_age() << endl;
```

```
    MeineSocke.set_age(3);
```

```
    cout << MeineSocke.get_age() << endl;
```

```
    return 0;
```

```
}
```

Pointer auf Objekte



```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

    ~Socke() {}

    int get_age() {
        return age;
    }

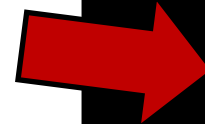
    void set_age(int new_age) {
        age = new_age;
    }
}
```

```
private:
    int age;        // sock age in years
    int size;       // size EU
    int value;      // in EUR
};

int main(){
    Socke MeineSocke;
    cout << MeineSocke.get_age() << endl;
    MeineSocke.set_age(3);
    cout << MeineSocke.get_age() << endl;

    Socke* ptr_Socke; // Pointer auf Socke
    ptr_Socke = &MeineSocke;
    cout << ptr_Socke->get_age() << endl;

    return 0;
}
```



Pointer auf Objekte



```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

    ~Socke() {}

    int get_age() {
        return age;
    }

    void set_age(int new_age) {
        age = new_age;
    }
}
```

```
private:
    int age;        // sock age in years
    int size;       // size EU
    int value;      // in EUR
};

int main(){
    Socke MeineSocke;
    cout << MeineSocke.get_age() << endl;
    MeineSocke.set_age(3);
    cout << MeineSocke.get_age() << endl;

    Socke* ptr_Socke; // Pointer auf Socke
    ptr_Socke = &MeineSocke;
    cout << ptr_Socke->get_age() << endl;

    return 0;
}
```

Syntax -> wie bei Zeiger
auf struct



Überladung von Funktionen



```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

    Socke(int p_age, int p_size, int p_value)
    {
        age = p_age;
        size = p_size;
        value = p_value;
    }

    ~Socke() {}
}
```

Überladener Konstruktor

[..]

„Werden in einem Programm mehrere Funktionen gleichen Namens, aber mit unterschiedlicher Parameterliste definiert, spricht man von **Überladung**“ [1]

Beide Funktionen sind eindeutig unterscheidbar, da die Signaturen der Funktionen (Name + Parameterliste) unterschiedlich sind.



Überladung von Funktionen



„Werden in einem Programm mehrere Funktionen gleichen Namens, aber mit unterschiedlicher Parameterliste definiert, spricht man von **Überladung**“ [1]

Beide Funktionen sind eindeutig unterscheidbar, da die Signaturen der Funktionen (Name + Parameterliste) unterschiedlich sind.

The prototypes of pow() as defined in the cmath header file are:

```
double pow(double base, double exponent);  
float pow(float base, float exponent);  
long double pow(long double base, long double exponent);
```

Überladung von Funktionen



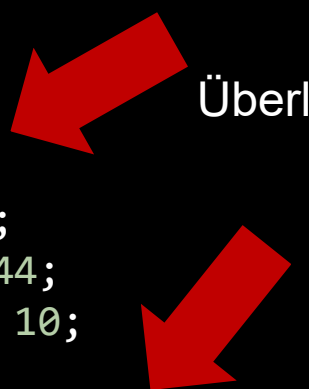
```
#include <iostream>

class Socke
{
public:
    Socke()
    {
        age = 0;
        size = 44;
        value = 10;
    }

    Socke(int p_age, int p_size, int p_value)
    {
        age = p_age;
        size = p_size;
        value = p_value;
    }

    ~Socke() {}
}
```

Überladener Konstruktor



[..]

„Werden in einem Programm mehrere Funktionen gleichen Namens, aber mit unterschiedlicher Parameterliste definiert, spricht man von **Überladung**“ [1]

Beide Funktionen sind eindeutig unterscheidbar, da die Signaturen der Funktionen (Name + Parameterliste) unterschiedlich sind.

```
int main(){
    Socke MeineSocke;
    Socke MeineZweiteSocke(12,38,0)

    return 0;
}
```





- Kommerzielle Software enthält eine Vielzahl unterschiedlicher "Geschäftsobjekte".
- Um die Übersicht zu behalten und damit mehrere Teammitglieder gleichzeitig Änderungen machen können, wird der Source-Code modularisiert und auf mehrere .cpp-Dateien aufgeteilt. In den einzelnen .cpp-Dateien werden dann nur die Methoden einer einzelnen Klasse definiert.
- Damit andere Teile des Programms diese Definitionen nutzen können, müssen die Objekte (Eigenschaften und Methoden) jeweils als Deklaration eingebunden werden können.
- Diese Aufgabe erlauben sog. "Header-Dateien" (Kopf-Datei), die jeweils nur die Deklaration durchführen.
- Der Compiler übersetzt dann alle einzelnen .cpp-Dateien; der Linker fügt dann die Übersetzungen zu einem Programm zusammen.

Modulare Programmierung



main.cpp

```
#include "CKonto.h"

int main()
{
    CKonto myKonto;
    myKonto.Einzahlen(15);
    double betrag =
        myKonto.Abheben(10);
}
```

CKonto.h

```
class CKonto
{
public:
    CKonto();
    double Saldo();
    void Einzahlen(double pBetrag);
    double Abheben(double pBetrag);

private:
    double mBetrag;
};
```

CKonto.cpp

```
#include "CKonto.h"

CKonto::CKonto()
{..}

double
CKonto::Abheben(double pBetrag)
{..}

[..]
```

Modulare Programmierung



main.cpp

```
#include "CKonto.h"

int main()
{
    CKonto myKonto;
    myKonto.Einzahlen(15);
    double betrag =
        myKonto.Abheben(10);
}
```

CKonto.h

```
class CKonto
{
public:
    CKonto();
    double Saldo();
    void Einzahlen(double pBetrag);
    double Abheben(double pBetrag);
    double mBetrag;
};
```

CKonto.cpp

```
#include "CKonto.h"

CKonto::CKonto()
{..}

CKonto::Abheben(double pBetrag)
{..}

[...]
```

Nächste Woche mehr hierzu...



Code aus der Vorlesung



```
// Ueberladen von Funktionen
void test() {
    cout << "test" << endl;
}

void test(int Input) {
    cout << "ueberladener test " << endl;
}

int main() {
    test(321321);
    return 0;
}
```



Code aus der Vorlesung



```
#include <iostream>
using namespace std;

// Alle Attribute & Methoden einer Klasse sind die "Member" der Klasse
class Tresor {
private:
    // Attribute (Eigenschaften)
    int Code = 1234;
    string Farbe = "schwarz";
public:
    Tresor() { // Konstruktor
        Farbe = "rot";
        cout << "Ein Objekt der Klasse Tresor wurde erzeugt...\n";
    }
    ~Tresor() { // Destructor
        cout << "Tresor Objekt existiert nicht mehr\n";
    }
    // Attribute
    bool schmutzig = false;
    bool AenderCode(int NeuerCode, int Chef_passwd) {
        if (Chef_passwd == 42) {
            cout << "Code konnte geaendert werden\n";
            Code = NeuerCode;
            return true;
        }
        else {
            cout << "Eingabe falsch - ALARM!!!\n";
            return false;
        }
    }
}
```

```
// Methoden (Funktion)
bool OeffneDich(int Eingabe) {
    if (Eingabe == Code) {
        cout << "Eingabe korrekt - Tresor geht auf!\n";
        return true;
    }
    else {
        cout << "Eingabe falsch - ALARM!!!\n";
        return false;
    }
};

int main() {
    Tresor MeinTresor; // MeinTresor ist eine "Instanz der Klasse
                        // Tresor" oder ist ein "Objekt" der Klasse Tresor

    Tresor Schliessfaecher[2000];
    Schliessfaecher[0].AenderCode(100, 42);
    Schliessfaecher[5].schmutzig = true;

    //MeinTresor.Code = 3333;
    MeinTresor.OeffneDich(1234);

    Tresor* MeinTresorPtr = new Tresor;
    delete MeinTresorPtr;
    return 0;
}
```

Code aus der Vorlesung

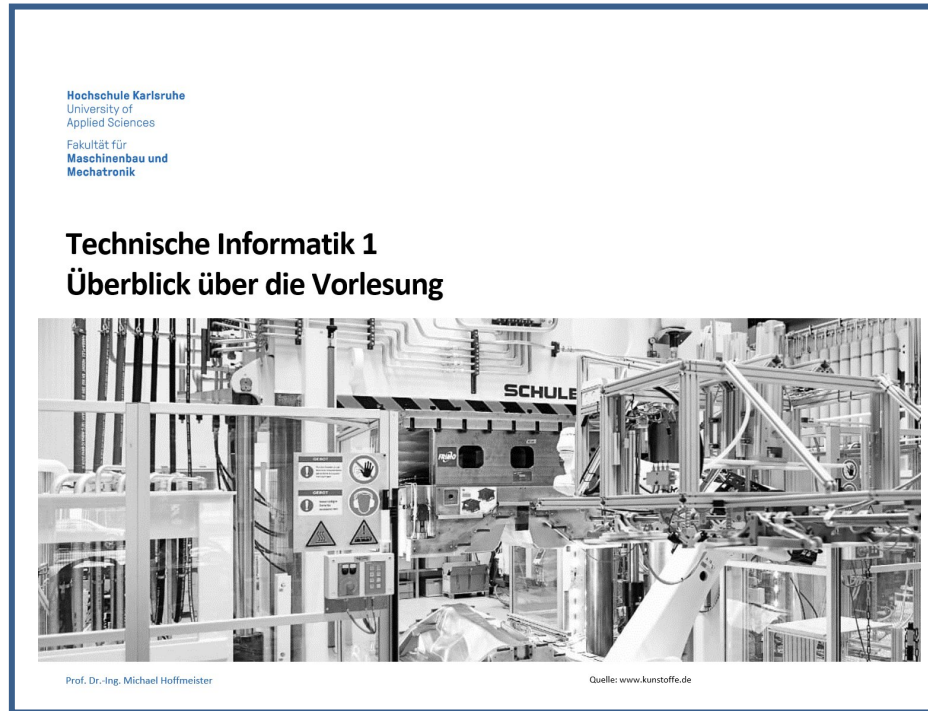
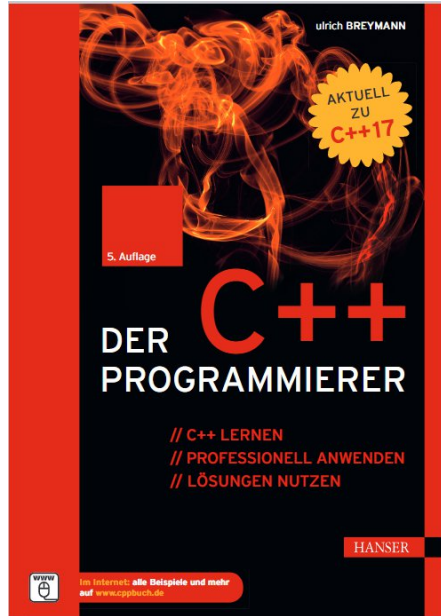


```
#include <iostream>
using namespace std;
// Alle Attribute & Methoden einer
// Klasse sind die "Member" der Klasse
class Kuh {
private:
    string Farbe = "braun";
    int AnzahlBeine = 4;
    int AnzahlOhren = 2;
    void Muhen() { cout << "MUHHHH\n"; }
public:
    Kuh() { // Konstruktor
    };
    Kuh(string AndereFarbe) { // Konstruktor
        Farbe = AndereFarbe;
    };

    ~Kuh() { // Destruktor
        Muhen();
    };
    string get_Farbe() {
        return Farbe;
    }
    void set_Farbe(string Input) {
        Farbe = Input;
    }
};
```

```
int main() {
    Kuh Elsa, Vera, Anja;
    cout << Elsa.get_Farbe() << endl;
    Vera.set_Farbe("PINK");
    Kuh MilkaKuh("lila");
    cout << MilkaKuh.get_Farbe() << endl;

    Kuh* PtrAufKuh = nullptr;
    PtrAufKuh = &Vera;
    cout << PtrAufKuh->get_Farbe() << endl;
    PtrAufKuh = &Elsa;
    cout << PtrAufKuh->get_Farbe() << endl;
}
```



Prof. Dr.-Ing. Michael Hoffmeister

E2B240 Grundlagen der Informatik 2 - Christian Langen

Grundlagen der Informatik 2

Prof. Dr.-Ing. Christian Langen

Hochschule Karlsruhe –Technik und Wirtschaft
University of Applied Sciences
Fakultät für Elektro- und Informationstechnik (EIT)
Studiengang Elektrotechnik – Automatisierungstechnik (Bachelor) EAB

Sommersemester 2013

Prof. Dr.-Ing. Christian Langen

Wo stehen wir?



Status Quo

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie `top()`, `pop()` und `push()` bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.

Wo stehen wir?



Status Quo

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie `top()`, `pop()` und `push()` bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.



Prozedurales
Programmieren

Welche Probleme gibt es?



Status Quo

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie `top()`, `pop()` und `push()` bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.

Probleme

- Was passiert, wenn viele verschiedene Datentypen benötigt werden?
- Was passiert, wenn diese Datentypen Ähnlichkeiten aber auch Unterschiede aufweisen?
- Was passiert, wenn es dann viele Funktionen auf vielen Datentyp gibt?
- Was passiert, wenn unterschiedliche Personen gleichzeitig an großen Projekten arbeiten?

Welche Probleme gibt es?



Status Quo

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie top(), pop() und push() bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.

Probleme

- Was passiert, wenn viele verschiedene Datentypen benötigt werden? → Die **Komplexität steigt!**
- Was passiert, wenn diese Datentypen Ähnlichkeiten aber auch Unterschiede aufweisen? → Die Gefahr von "Copy&Paste" steigt, die **Wartbarkeit sinkt!**
- Was passiert, wenn es dann viele Funktionen auf vielen Datentyp gibt? → Die **Zuordnung** zwischen Datentypen und entsprechenden Funktionen **fällt schwer!**
- Was passiert, wenn unterschiedliche Personen gleichzeitig an großen Projekten arbeiten? → Die **Änderungen** durch eine Person können andere Personen **behindern!**

Wie weit kommen wir damit?



Status Quo

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie `top()`, `pop()` und `push()` bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.

- Bis wann glauben Sie war dieser Ansatz ausreichend?
- Und was änderte sich zu diesem Zeitpunkt?

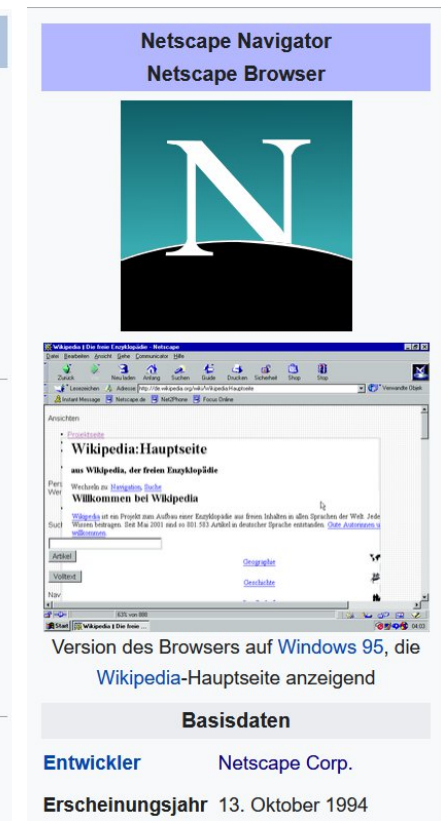
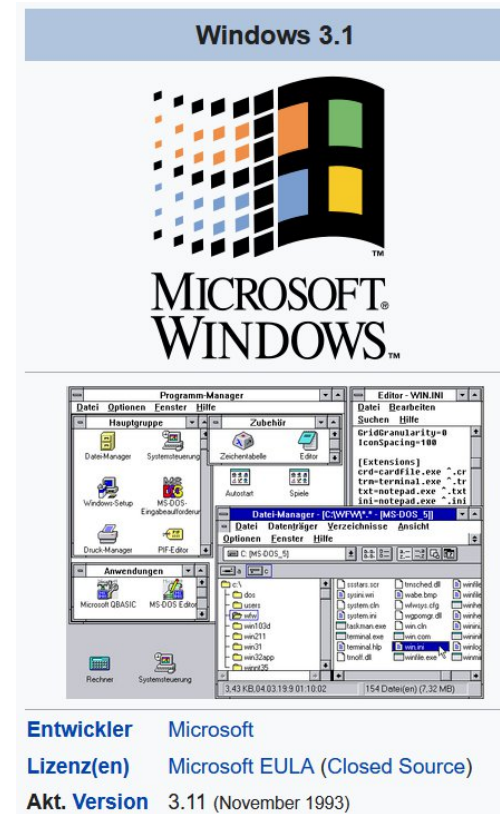
Wie weit kommen wir damit?



Status Quo

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie `top()`, `pop()` und `push()` bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.

~1990er Jahre



Wie weit kommen wir damit?



Status Quo

- **Strukturierte Datentypen** erlauben es einzelne Datenelemente zu Sinneinheiten zusammenzustellen.
- Mit Hilfe von **Datenfeldern** und **dynamischen Speicher** lassen sich viele von diesen Sinneinheiten allozieren (belegen) und nacheinander bearbeiten.
- So konnten zum Beispiel **Sortieralgorithmen** auf diesen Feldern arbeiten und **komplexe Funktionen** einfach zur Verfügung stellen.
- Auch wurden **dynamische Datenstrukturen** vorgestellt, die **weitere Funktionen** wie `top()`, `pop()` und `push()` bereitstellen, aber jeweils von der Applikation auf die **passenden strukturierte Datentypen** angewendet werden mussten.

~1990er Jahre

- Tendenz zur Verwendung vernetzter Geräte: Verwendung von grafischen Betriebssystemen & Benutzeroberflächen, Netzwerk-Stacks (TCP/IP).
- Notwendige Softwarefunktionalität ist bereits in Bibliotheken vorhanden und muss daher nicht mehr neu implementiert werden.
- Verwendung dieser Bibliotheken als „Baukastensystem“ Bausteine sind Objekte, die auch dann verwendet werden können, wenn man deren Inhalt nicht kennt, sondern nur Merkmale und Funktionsweise



- **Klassen** sind strukturierte Datentypen, die **Attribute**/Eigenschaften (Variablen) und **Methoden** (Funktionen) zusammenfassen.
- Die Instanzen eines solchen Datentyps werden **Objekte** (oder auch Instanzen) genannt.
- Die einzelnen Attribute und Methoden werden **Mitglied** (engl. **Member**) der Klasse genannt.



Vorteile der OOP Denkweise



Ganzheitliche Einheiten:

- Keine Trennung von Daten und Funktionen mehr!

Bessere Abstraktionsmöglichkeit (Vorteil? – ja!):

- Weniger problemorientiert als lösungsorientiert.

Methodische Durchgängigkeit:

- Die selben Konzepte in allen Phasen der Entwicklung.
- Identisches Verhalten ähnlicher Objekte vereinfacht Programmierung.

Evolutionäre Entwicklung:

- Zyklische Erweiterung von Programmen.
- Zunehmende Komplexität unter Weiterverwendung vorhandener Klassen möglich.

Praktische Vorteile von OOP



- Spätere Anforderungen können noch leicht nachträglich integriert werden.
- Anspruchsvollere Aufgaben lösbar.
- Bessere **Erweiterbarkeit**
- Bessere **Wiederverwendbarkeit**
- Leichtere Dokumentation
- Näher an der **menschlichen Denkweise**



Vorgehensweise bei OOP:

- Identifizieren der Klassen und Objekte (das Schwierigste!).
- Individuelle Objekte klassifizieren, gemeinsame Eigenschaften herausarbeiten.
- Zuweisen der Attribute und Methoden.
- Identifizieren der Beziehungen:
 - „Ist ein“ -> Vererbung*
 - „Hat ein“ -> Assoziation*

Herkömmliche Entwurfsmethoden (Wasserfallmodell) sind vollkommen ungeeignet, da bei OOP grundlegend anderer Ansatz.

Was ist OOP?



1. Everything is an object,



Abstraktionen der Realität

Was ist OOP?



1. Everything is an object



Abstraktionen der Realität



Was ist OOP?



1. Everything is an object



Abstraktionen der Realität



Datentyp `string`

Was ist OOP?



1. Everything is an object



Abstraktionen der Realität



Datentyp `string`

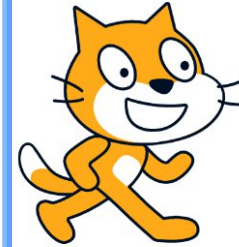
Was ist OOP?

1. Everything is an object
2. Objects communicate by sending and receiving messages



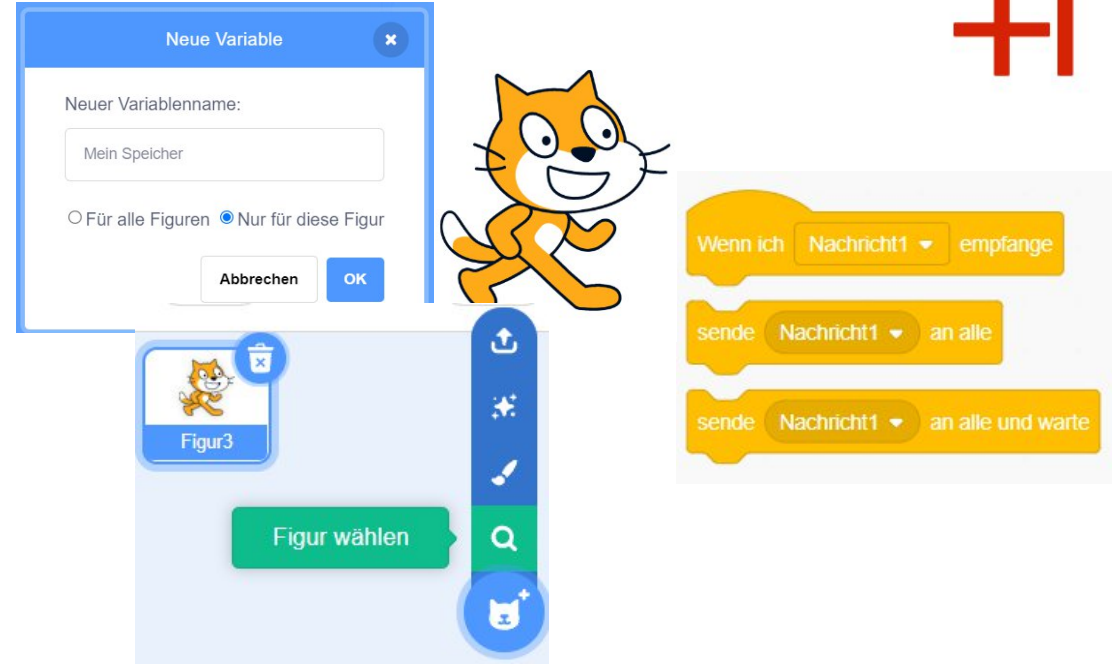
Was ist OOP?

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory



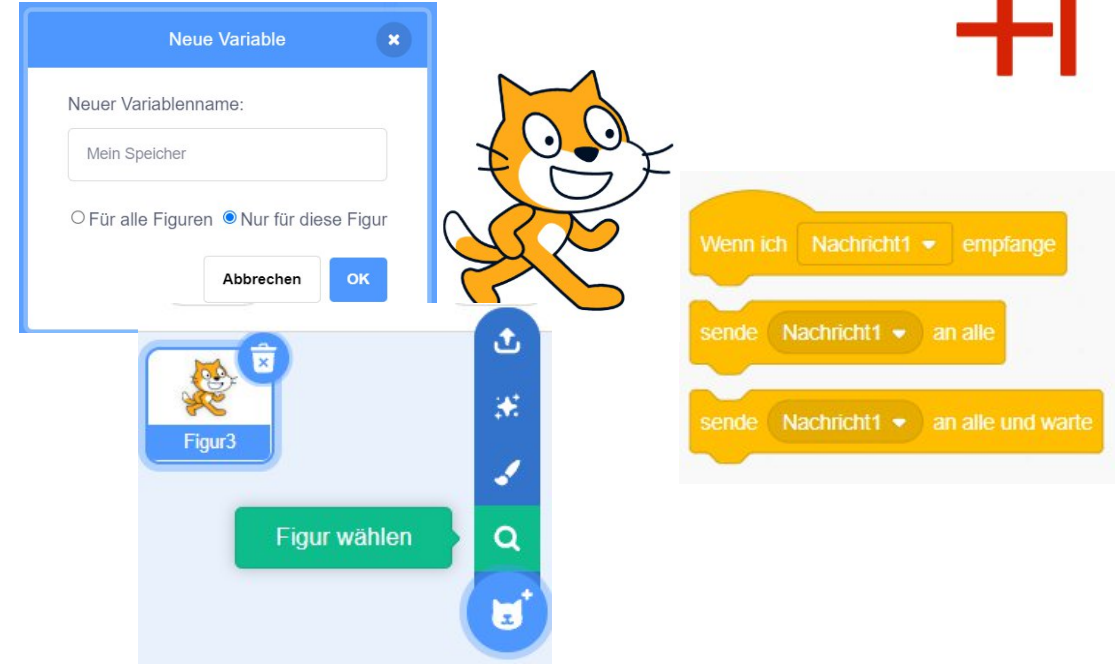
Was ist OOP?

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class



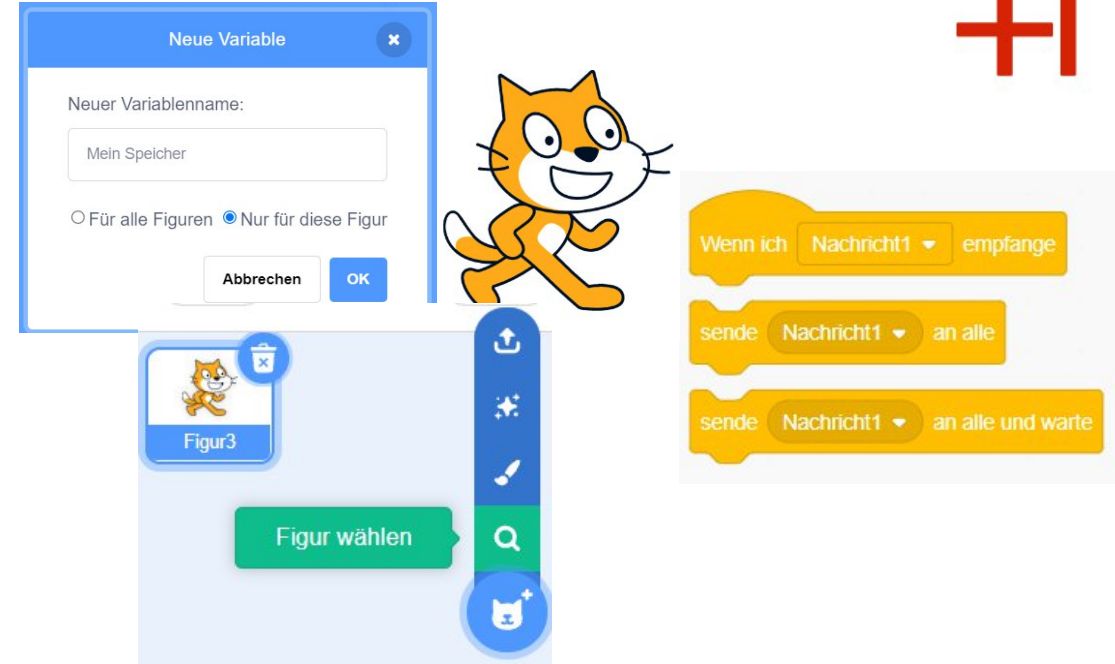
Was ist OOP?

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class
5. The class holds the shared behavior for its instances



Was ist OOP?

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class
5. The class holds the shared behavior for its instances
6. Program control is passed to the object

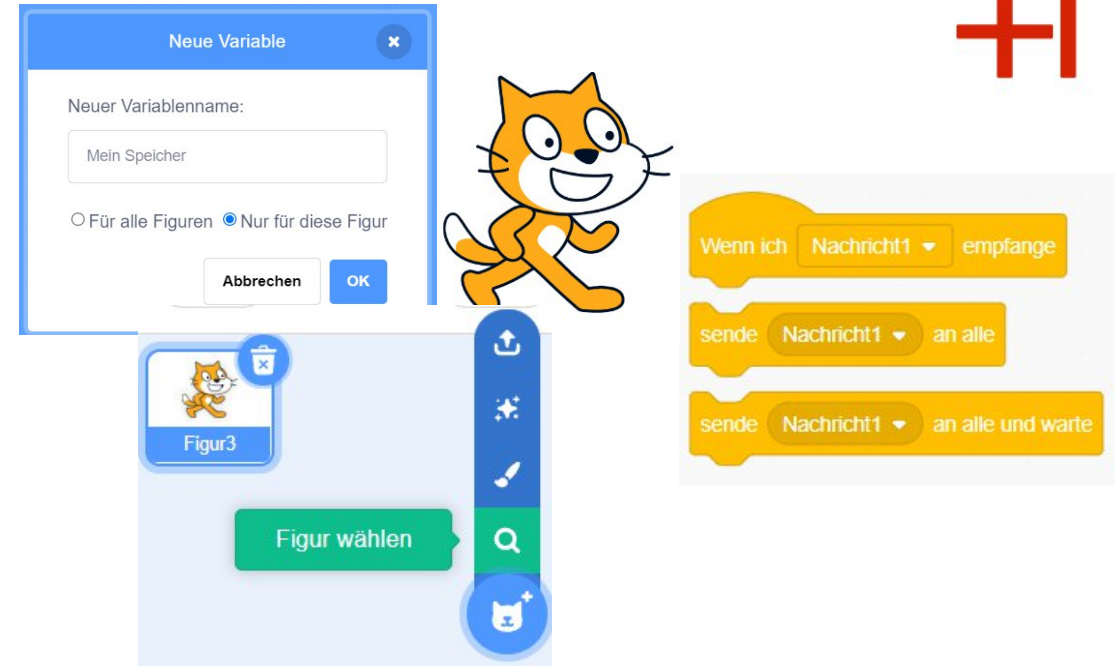


Was ist OOP?

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class
5. The class holds the shared behavior for its instances
6. Program control is passed to the object

Allgemein: **Jedes Objekt im System** kann als ein **abstraktes Modell eines Akteurs** betrachtet werden,

- der Aufträge erledigen,
- seinen Zustand berichten und ändern
- und mit den anderen Objekten im System kommunizieren kann
- ohne offenlegen zu müssen, wie diese Fähigkeiten implementiert.



Was ist OOP?

1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class
5. The class holds the shared behavior for its instances
6. Program control is passed to the object

Allgemein: Jedes Objekt im System kann als ein abstraktes Modell eines Akteurs betrachtet werden,

- der Aufträge erledigen,
- seinen Zustand berichten und ändern
- und mit den anderen Objekten im System kommunizieren kann
- ohne offenlegen zu müssen, wie diese Fähigkeiten implementiert.

Solche Abstraktionen sind Klassen in der klassenbasierten Objektorientierung.

